2022

SISTEMAS EN CHIP

PROYECTO 2 “PROYECTO 02 BEE-BOT”

FRANCISCO UZIEL CORDOVA PICHARDO

Objetivo

Al término de la sesión, los integrantes del equipo contaran con la habilidad de hacer uso de la memoria EEPROM del microcontrolador..

Desarrollo Experimental

1.- El objetivo de éste proyecto es diseñe un móvil programable el cual será capaz de grabarle una secuencia y después

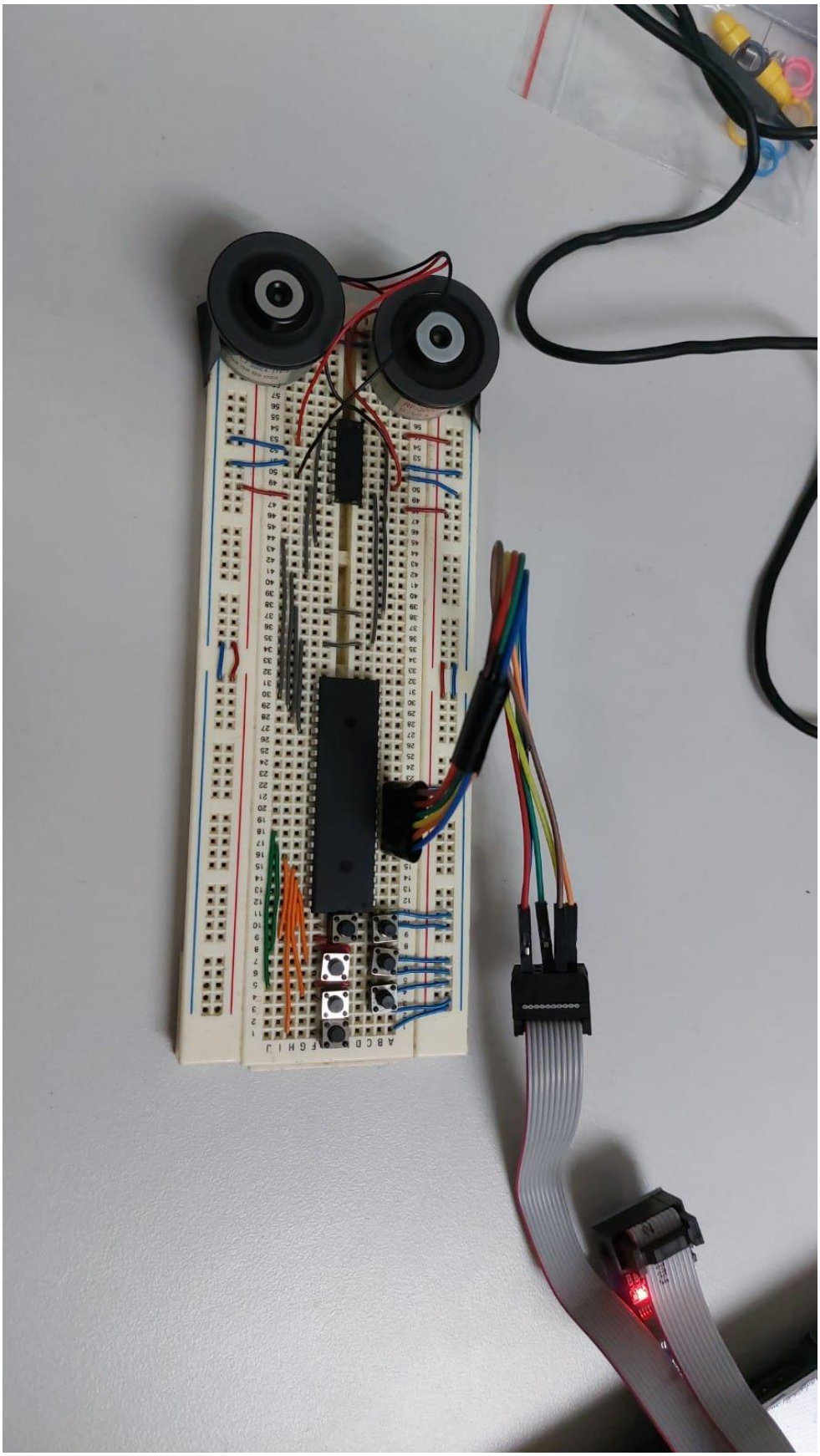
reproducirla. Con el botón CLEAR se borrará la última secuencia y estará listo para ingresar la nueva con los botones de

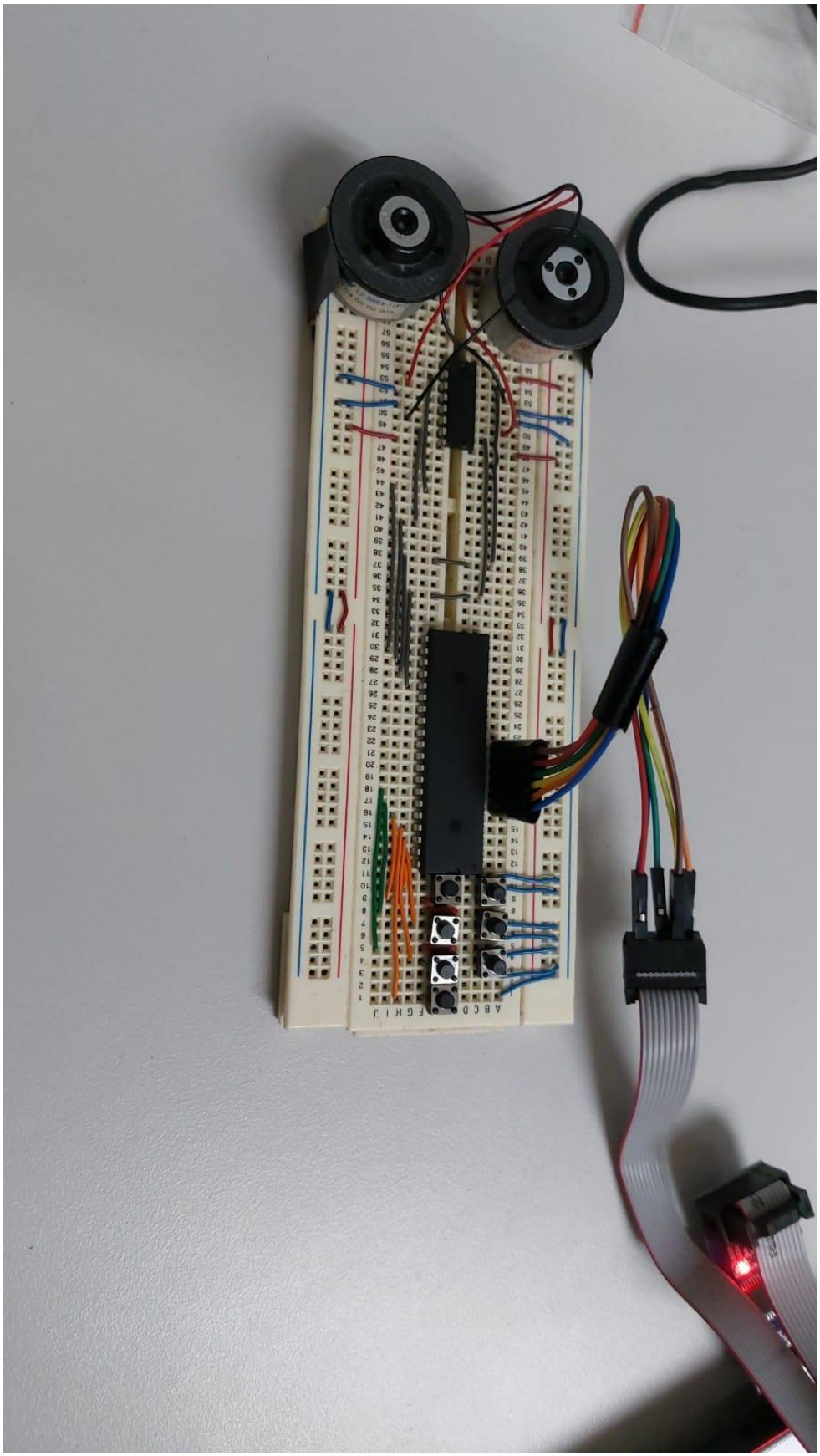
FLECHAS (ustedes determinen hasta cuantas secuencias podrán programas, ejemplo 10, 20, 30, etc.). Para iniciar la

secuencia deberá oprimir el botones GO y para detenerse el botón PAUSE o terminar la secuencia previamente

grabada.

Fotografías del circuito funcionando





Código del programa

```
/*****************************************************
This program was created by the CodeWizardAVR V3.32
Automatic Program Generator
© Copyright 1998-2017 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project :
Version :
Date    : 13/01/2023
Author  :
Company :
Comments:


Chip type               : ATmega8535
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 128
*****************************************************/

#include <mega8535.h>
#include <delay.h>
#define IZQ PINA.0
#define DER PINA.1
#define ARR PINA.2
#define ABA PINA.3
#define CLR PINA.4
#define PAU PINA.5
#define GO PINA.6
#define ARRIBA 0x0A
#define ABAJO  0x05
#define IZQI 0x02
#define DERE 0x08

const unsigned int size = 10;
unsigned char read_index = 0, write_index = 0;
unsigned char inst[size];
bit a_izq, a_der, a_arr, a_aba, a_clr, a_pau, a_go, p_izq,
    p_der, p_arr, p_aba, p_clr, p_pau, p_go, mode = 0;
```

```
// Declare your global variables here

void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) |
(0<<DDA1) | (0<<DDA0);
// State: Bit7=P Bit6=P Bit5=P Bit4=P Bit3=P Bit2=P Bit1=P Bit0=P
PORTA=(1<<PORTA7) | (1<<PORTA6) | (1<<PORTA5) | (1<<PORTA4) | (1<<PORTA3) |
(1<<PORTA2) | (1<<PORTA1) | (1<<PORTA0);

// Port B initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out
Bit0=Out
DDRB=(1<<DDB7) | (1<<DDB6) | (1<<DDB5) | (1<<DDB4) | (1<<DDB3) | (1<<DDB2) |
(1<<DDB1) | (1<<DDB0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) |
(0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out
Bit0=Out
DDRC=(1<<DDC7) | (1<<DDC6) | (1<<DDC5) | (1<<DDC4) | (1<<DDC3) | (1<<DDC2) |
(1<<DDC1) | (1<<DDC0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) |
(0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out
Bit0=Out
DDRD=(1<<DDD7) | (1<<DDD6) | (1<<DDD5) | (1<<DDD4) | (1<<DDD3) | (1<<DDD2) |
(1<<DDD1) | (1<<DDD0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) |
(0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// Timer/Counter 0 initialization
// Clock source: System Clock
// Clock value: Timer 0 Stopped
```

```
// Mode: Normal top=0xFF
// OC0 output: Disconnected
TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) |
(0<<CS01) | (0<<CS00);
TCNT0=0x00;
OCR0=0x00;

// Timer/Counter 1 initialization
// Clock source: System Clock
// Clock value: Timer1 Stopped
// Mode: Normal top=0xFFFF
// OC1A output: Disconnected
// OC1B output: Disconnected
// Noise Canceler: Off
// Input Capture on Falling Edge
// Timer1 Overflow Interrupt: Off
// Input Capture Interrupt: Off
// Compare A Match Interrupt: Off
// Compare B Match Interrupt: Off
TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) |
(0<<WGM10);
TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) |
(0<<CS11) | (0<<CS10);
TCNT1H=0x00;
TCNT1L=0x00;
ICR1H=0x00;
ICR1L=0x00;
OCR1AH=0x00;
OCR1AL=0x00;
OCR1BH=0x00;
OCR1BL=0x00;

// Timer/Counter 2 initialization
// Clock source: System Clock
// Clock value: Timer2 Stopped
// Mode: Normal top=0xFF
// OC2 output: Disconnected
ASSR=0<<AS2;
TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) |
(0<<CS21) | (0<<CS20);
TCNT2=0x00;
OCR2=0x00;

// Timer(s)/Counter(s) Interrupt(s) initialization
```

```
TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) |
(0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);

// External Interrupt(s) initialization
// INT0: Off
// INT1: Off
// INT2: Off
MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
MCUCSR=(0<<ISC2);

// USART initialization
// USART disabled
UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) |
(0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

// Analog Comparator initialization
// Analog Comparator: Off
// The Analog Comparator's positive input is
// connected to the AIN0 pin
// The Analog Comparator's negative input is
// connected to the AIN1 pin
ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) |
(0<<ACIS1) | (0<<ACIS0);
SFIOR=(0<<ACME);

// ADC initialization
// ADC disabled
ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) |
(0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);

// SPI initialization
// SPI disabled
SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) |
(0<<SPR1) | (0<<SPR0);

// TWI initialization
// TWI disabled
TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

while (1)
      {
        char current;
        char i;

        // Botones
```

```
        a_clr = CLR;
        a_pau = PAU;
        a_go = GO;

        if (p_clr == 1 && a_clr == 0) {
            for (i=0; i<size; i++) inst[i] = 0;
            write_index = 0;
            read_index = 0;
            mode = 0;
            delay_ms(20);
        }

        if (p_pau == 1 && a_pau == 0) {
            mode = 0;
            delay_ms(20);
        }

        if (p_go == 1 && a_go == 0) {
            mode = 1;
            delay_ms(20);
        }

        if (p_clr == 0 && a_clr == 1) delay_ms(20);
        if (p_pau == 0 && a_pau == 1) delay_ms(20);
        if (p_go == 0 && a_go == 1) delay_ms(20);

        // Instrucciones: grabar y leer
        if (mode == 0 && write_index < size) { // Puede grabar
            a_izq = IZQ;
            a_der = DER;
            a_arr = ARR;
            a_aba = ABA;

            if (p_izq == 1 && a_izq == 0) {
                inst[write_index] = 1;
                write_index++;
                delay_ms(20);
            }

            if (p_der == 1 && a_der == 0) {
                inst[write_index] = 2;
                write_index++;
                delay_ms(20);
            }
```

```
            if (p_arr == 1 && a_arr == 0) {
                inst[write_index] = 3;
                write_index++;
                delay_ms(20);
            }

            if (p_aba == 1 && a_aba == 0) {
                inst[write_index] = 4;
                write_index++;
                delay_ms(20);
            }

            if (p_izq == 0 && a_izq == 1) delay_ms(20);
            if (p_der == 0 && a_der == 1) delay_ms(20);
            if (p_arr == 0 && a_arr == 1) delay_ms(20);
            if (p_aba == 0 && a_aba == 1) delay_ms(20);
        } else if (mode == 1 && read_index < size) {
            current = inst[read_index];
            read_index++;

            switch (current) {
               case 1: // Derecha
                PORTC = DERE;
                delay_ms(500);
                PORTC = 0x00;
                break;

               case 2: // Izquierda
                PORTC = IZQI;
                delay_ms(500);
                PORTC = 0x00;
                break;

               case 3: // Arriba
                PORTC = ARRIBA;
                delay_ms(500);
                PORTC = 0x00;
                break;

               case 4: // Abajo
                PORTC = ABAJO;
                delay_ms(500);
                PORTC = 0x00;
                break;
            }
```

```
        }

        p_izq = a_izq;
        p_der = a_der;
        p_arr = a_arr;
        p_aba = a_aba;
        p_clr = a_clr;
        p_pau = a_pau;
        p_go = a_go;

      }
}
```

Conclusiones

En conclusión este proyecto es interesante porque vemos otros usos que se le pueden dar a los microcontroladores y como estos pueden accionar secuencias utilizando la memoria para registrar los movimientos y poder accionarlos mandando las señales correctas.

Bibliografía

Manual del ATmega8535: https://pdf1.alldatasheet.es/datasheet-pdf/view/164169/ATMEL/ATMEGA8535.html